QA DOCUMENTATION / MAY 2026

MANUAL TESTING ASSESSMENT

WRITEBACK JOURNAL DRAFT

Quality assurance analysis for a custom Excel function that creates draft GL journal batches for manual review.

50	28	32	10
TEST CASES	TEST AREAS	SPEC GAPS	RISK MAP

Submission package

Review document in PDF format plus a separate Excel workbook containing the complete test case suite.

LIGHT EDITION
EXPANDED VIEW

// DOCUMENT MAP

CONTENTS

01 OVERVIEW

Feature understanding and source acceptance criteria

50	28	32	10
TEST CASES	TEST AREAS	SPEC GAPS	RISK MAP

Function Overview

`WRITEBACKJOURNALDRAFT` is a custom Excel function that creates **draft GL (General Ledger) journal batches** in an ERP system directly from a spreadsheet. Each function call defines a single journal line. Multiple calls on the same worksheet together form one **batch**. The function never posts - it always lands in **On Hold** status, requiring manual review inside the ERP.

> **Key invariant:** Total debits across all lines must equal total credits. An imbalanced batch is rejected before creation.

Function Signature

```
=WRITEBACKJOURNALDRAFT(
  // batch properties
  connectionName,
  date,
  reportingPeriod,
  batchDescription,
  branch,
  Ledger,
  // line properties
  currency,
  account,
  debit,
  credit,
  lineDescription
)
```

Parameters

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`connectionName`	string	YES	ERP connection name
`transactionDate`	date	YES	Date of the journal batch

PARAMETER	TYPE	REQUIRED	DESCRIPTION
`postPeriod`	string	YES	Financial period, e.g. `"05-2026"`
`batchDescription`	string	NO	Batch-level description
`branch`	string	NO	Branch identifier
`ledger`	string	YES	Target ledger in the ERP
`currency`	string	NO	Currency code; defaults to ledger's base currency
`account`	string	YES	GL account number for this line
`debit`	number	NO	Debit amount for this line
`credit`	number	NO	Credit amount for this line
`lineDescription`	string	NO	Description for this specific line

◇ Acceptance Criteria

✓ On Hold Status

Created batch always has status **On Hold** - no release or post action triggered.

⊖ Balance Validation

Total debits = total credits. Error: `"Journal is not balanced: debits X.XX ≠ credits Y.YY"`

≥1 Line Required

Empty range returns `"At least one transaction line is required"`.

↩ Return Value

On success: `"Draft batch #JE-00087 created"` (batch ID varies).

□ Glossary

Batch
: A group of journal lines sharing the same transaction date, ledger, and batch description.

Reporting Period
: A predefined accounting period the transaction date falls within - e.g. "Month Ending 31 March 2027".

On Hold
: A batch status - saved but not yet reviewed or posted. Posting is done manually inside the ERP.

02 SPEC REVIEW

Expanded ideal specification, field contract and financial risk map

◇ Ideal Specification - What Is Missing

A complete specification for this feature should include all of the following sections. Sections absent from the current spec are the source of the 32 gaps documented in the Spec Gaps page.

01 Business goal and user story

Defines who uses the feature and why a draft is created instead of an immediately posted transaction.

EXAMPLE

As an ERP user, I prepare GL journals in Excel for accountant review; writeback always creates an unposted draft.

02 Definitions: batch, line, ledger, postPeriod, debit, credit, On Hold

Removes interpretation differences between a tester, developer and accountant.

EXAMPLE

A batch is a group of lines with one grouping key; *On Hold* means created but not released or posted.

03 Processing model: explicit Writeback vs. formula recalculation

Clarifies the event that sends data to ERP, which is essential for duplicate prevention.

EXAMPLE

Recalculating or reopening the workbook must not create a batch; only the explicit Writeback action sends the request.

04 Excel input rules and accepted date/number formats

Defines what is valid before ERP validation and avoids locale-related defects.

EXAMPLE

`DATE(2026,5,19)` is accepted and sent as `2026-05-19`; ambiguous text dates are rejected.

05 Normalized REST/OData/API payload and response contract

States exactly what the add-in sends and what success or failure response it can display.

EXAMPLE

Success response contains `batchNumber`, `status=OnHold` and `correlationId`.

06 ERP field mapping and validation responsibilities

Shows where each Excel value lands and whether Excel or ERP owns validation.

EXAMPLE

`ledger` maps to ERP Ledger ID; existence and access permissions are checked server-side.

07 Batch grouping rules - full grouping key definition

Prevents lines for different entities from being silently combined into one journal batch.

EXAMPLE

Lines are grouped only when connection, date, period, branch, ledger, currency and description all match.

08 Line-level debit/credit rules - mutual exclusion

Defines valid accounting shape of a single line.

EXAMPLE

Each line must contain a positive debit or a positive credit, never both and never neither.

09 **Balance validation scope - per batch, not global**

Makes clear that one imbalanced group cannot be hidden by another compensating group.

EXAMPLE
Batch A debit 100 / credit 90 is rejected even if Batch B has debit 90 / credit 100.

10 **Idempotency and duplicate prevention**

Protects financial data when a user retries after a delay or lost response.

EXAMPLE
The same operation ID returns the original batch number and never creates a second batch.

11 **Atomicity and partial failure behavior**

Specifies whether all lines must be saved together and how failures roll back.

EXAMPLE
If line 4 has an invalid account, zero header or line records remain in ERP.

12 **Timeout/retry behavior and safe error messages**

Avoids misleading success or blind retries when ERP state is unknown.

EXAMPLE
On timeout, display an operation ID and advise status lookup; retry uses the same idempotency key.

13 **Permissions model and access control**

Determines which users may write and which ledgers or branches they may target.

EXAMPLE
A user without Create Draft Journal permission receives an authorization error and no ERP record.

14 **Telemetry, audit trail, and correlation ID**

Supports investigation of financial operations without exposing sensitive content.

EXAMPLE
Audit stores user, timestamp, operation ID, batch number and outcome, excluding credentials.

15 **Cross-platform support: Windows, macOS, Excel Web**

States environments in which the function is expected to behave consistently.

EXAMPLE
Windows and macOS desktop are supported; Excel Web either works identically or shows a supported-platform message.

16 **Error codes, messages, and validation priority order**

Makes failures predictable when a row violates several rules at once.

EXAMPLE
Required-field errors appear before balance checks and identify the field and Excel row.

17 **Field length limits and numeric precision rules**

Defines boundaries for text and monetary values so no rounding or truncation is silent.

EXAMPLE
Amounts support two decimal places; over-precision is rejected rather than silently rounded.

18 **Out of scope and known limitations**

Prevents reviewers from expecting behavior the feature intentionally does not provide.

EXAMPLE
Posting, releasing, editing existing batches and exchange-rate calculation are outside this function.

Ideal Field Specification

Proposed field contract: how each parameter should be defined in a complete, production-ready feature specification. It connects Excel input, normalised API data, ERP mapping, validation, defaults, and limits.

01 / INPUT

02 / CONTROL

03 / OUTPUT

Spreadsheet values

Fields entered by the user: dates, ledger, account, currency, descriptions and debit/credit amounts.

Safe processing

System-generated operation and line identifiers used for duplicate prevention, tracing and safe retry.

Observable result

Success/error response, batch status, ERP lookup and audit evidence needed to confirm the writeback result.

QA REVIEW

Good foundation, but not sufficient for test execution without clarification.

Source / status
Mark each rule as original requirement, agreed clarification, or proposed control.

Error contract
Define code, message, affected row/field/group and validation order.

Validation ownership
State whether Excel add-in, API or ERP rejects each invalid value.

Processing rules
Identify grouping-key fields, idempotency scope and audit/security handling.

Requirement boundary: the user-input rows represent fields described in the task. `externalLineId`, `idempotencyKey` and `correlationId` are proposed reliability controls, not confirmed source requirements.

FIELD	SCOPE	REQUIRED	EXCEL FORMAT	API FORMAT	ERP MAPPING	VALIDATION RULE	DEFAULT	LIMIT
connectionName	Input / system	YES	Text - connection name	string	ERP connection selector	Must exist and be accessible	none	255 chars
transactionDate	Batch	YES	Excel date or `DATE(Y,M,D)`	YYYY-MM-DD	Transaction Date	Valid date; must belong to postPeriod	none	ISO date
postPeriod	Batch	YES	Text period code, e.g. `05-2026`	string / periodId	Financial Period	Exists in ERP and is open (not closed/locked)	none	Format defined by ERP
batchDescription	Batch	NO	Text	string	Batch Description	Safe text; part of grouping key	empty / null	255 chars
branch	Batch	NO / DEPENDS	Branch code	branch code / id	Branch	If provided: exists and accessible; part of grouping key	default / empty	30–50 chars
ledger	Batch	YES	Ledger code / name / ID - must be specified	ledger code / id	Ledger	Active and accessible ERP ledger	none	30–50 chars
currency	Batch / Line	NO	Currency code	currency code	Currency	Supported by ledger; consistent across all lines in batch	ledger base currency	3 chars (ISO 4217)
account	Line	YES	Account code	account code / id	GL Account	Active and accessible in chart of accounts	none	30–50 chars
debit	Line	Conditional	Excel number	decimal	Debit Amount	> 0; valid precision; only if credit is empty	null	decimal(19,4)
credit	Line	Conditional	Excel number	decimal	Credit Amount	> 0; valid precision; only if debit is empty	null	decimal(19,4)
lineDescription	Line	NO	Text	string	Line Description	Safe text; no script or markup execution	empty / null	255 chars
externalLineId	Line / system	Generated	Hidden / generated	string	Trace / source line	Unique within operation; links ERP line back to Excel row	auto-generated	36–255 chars

FIELD	SCOPE	REQUIRED	EXCEL FORMAT	API FORMAT	ERP MAPPING	VALIDATION RULE	DEFAULT	LIMIT
idempotencyKey	Operation / system	Generated	Hidden / sent as request header	string / uuid	Writeback operation	Unique per operation; prevents duplicate batches on retry	auto-generated	36–255 chars
correlationId	Operation / system	Generated	Hidden / generated	string / uuid	Trace / audit	Unique per operation; links Excel, API logs, telemetry, ERP audit	auto-generated	36–255 chars

Contracts still needed before QA sign-off

Response contract
Specify success payload and visible result, for example batch number, `On Hold` status and correlation ID.

Error catalogue
Specify messages/codes for required fields, balance, invalid master data, permissions and timeout states.

Grouping and atomicity
List the full batch key and whether one invalid group blocks all groups or only that group.

Retry and audit evidence
Define duplicate protection, safe retry outcome and what evidence is available in ERP/logs.

◈ Risk Map - Financial Impact

- **Duplicate batch creation**
  Excel recalculation, workbook reopen, or timeout retry creates multiple identical GL journal drafts - a critical financial data integrity risk.
- **Wrong status or auto-posting**
  Batch bypasses "On Hold" and gets posted/released automatically - journal entries appear in financial reports without accountant review.
- **Wrong balance validation scope**
  Global-range balance check masks imbalanced individual batches - incorrect financial data passes silently.
- **Partial batch creation (atomicity)**
  One invalid line leaves a partial batch in ERP - an imbalanced orphaned journal entry that is difficult to detect and reverse.
- **Wrong ledger / account / period**
  Financial data posted to incorrect ledger or period corrupts reporting and may require audit-trail investigation to reverse.
- **Date/locale conversion errors**
  Ambiguous text dates (05/06 = May 6 vs June 5) silently produce wrong accounting dates across platforms.
- **Permission bypass**
  Unauthorized financial writes if ERP-side permissions are not enforced server-side.
- **Missing traceability**
  No correlation ID or audit log entry means support cannot investigate ERP/API failures after they occur.
- **Cross-platform inconsistency**
  Different date or decimal behavior between Windows / macOS / Excel Web produces incorrect journal data depending on the user's environment.

- **Script / markup injection via text fields**
  Unsanitized batchDescription or lineDescription could execute in ERP UI - XSS or formula injection risk.

03 SPEC GAPS & ASSUMPTIONS

Complete list - all severity levels included

4	21	7	32
CRITICAL	HIGH	MEDIUM	TOTAL GAPS

G-01 HIGH

Parameter Names Are Inconsistent

WHAT IS UNCLEAR

The function signature uses `date` and `reportingPeriod`, but the parameter table uses `transactionDate` and `postPeriod` for the same fields. It is not clear which names should be used in documentation, error messages, and test cases.

ASSUMPTION

Use the parameter table names for testing: `transactionDate` and `postPeriod`. The product spec should choose one final naming standard.

WHY IT MATTERS

If different names are used in different places, users and testers may not understand which field failed validation.

TEST COVERAGE

Validation cases for required fields and error messages.

G-02 CRITICAL

It Is Not Clear When Data Is Sent To ERP

WHAT IS UNCLEAR

The spec does not clearly say whether data is sent to ERP when the formula recalculates, or only when the user starts a Writeback action.

ASSUMPTION

Data should be sent to ERP only when the user starts an explicit Writeback action. Normal Excel recalculation must not create or update anything in ERP.

WHY IT MATTERS

Excel can recalculate when the file opens, when a cell changes, or when the user presses F9. If recalculation writes to ERP, duplicate journal drafts may be created.

TEST COVERAGE

Excel recalculation, workbook reopen, and duplicate prevention cases.

G-03 CRITICAL

Idempotency Is Not Defined

WHAT IS UNCLEAR

The spec does not describe how the system prevents duplicate batches when the same operation is retried after timeout, workbook reopen, or user retry.

ASSUMPTION

The backend should use an idempotency key or operation ID. Retrying the same operation should return the original batch, not create a new one.

WHY IT MATTERS

Without idempotency, one network timeout can lead to two identical journal drafts in ERP.

TEST COVERAGE

Retry after timeout and duplicate prevention cases.

G-04 HIGH

Batch Grouping Rules Are Not Fully Clear

WHAT IS UNCLEAR

The spec says one batch is created from lines with the same transaction date, ledger, and batch description. It does not say whether `connectionName`, `postPeriod`, `branch`, or `currency` also split lines into different batches.

ASSUMPTION

A batch should be created only from lines that have the same `connectionName`, `transactionDate`, `postPeriod`, `batchDescription`, `branch`, `ledger`, and `currency`.

WHY IT MATTERS

If grouping is unclear, lines from different periods, branches, or currencies may be placed into the wrong batch.

TEST COVERAGE

Batch grouping and per-batch balance cases.

G-05 CRITICAL

Balance Check Scope Is Not Clear

WHAT IS UNCLEAR

The spec says balance is checked across all lines, but it does not say whether this means all selected lines globally or each grouped batch separately.

ASSUMPTION

Debit and credit totals should be checked separately for each batch.

WHY IT MATTERS

Two wrong batches can look balanced if their totals cancel each other globally. That is not safe for accounting.

TEST COVERAGE

Per-batch balance cases.

G-06 HIGH

Batch Or Voucher Balance Level Is Not Clear

WHAT IS UNCLEAR

Some ERP systems check balance at batch level, while others may check it at voucher or journal entry level. The spec does not say which level is correct.

ASSUMPTION

Use batch-level balance unless the ERP has a lower-level voucher rule that is documented.

WHY IT MATTERS

Testers need to know the exact level where debit and credit must match.

TEST COVERAGE

Balance and ERP mapping cases.

G-07 HIGH

Debit And Credit Rules Are Not Fully Defined

WHAT IS UNCLEAR

Debit and credit are optional in the field table, but the spec does not say whether one line can have both debit and credit, or neither of them.

ASSUMPTION

Each line should have exactly one side: either debit greater than zero or credit greater than zero. A line must not have both and must not have neither.

WHY IT MATTERS

Ambiguous debit and credit rules can create meaningless or wrong journal lines.

TEST COVERAGE

Debit and credit validation cases.

G-08 HIGH

Minimum Valid Line Rule Is Incomplete

WHAT IS UNCLEAR

The spec says at least one transaction line is required, but one line alone cannot create a balanced journal batch.

ASSUMPTION

A valid batch should have at least one debit line and at least one credit line.

WHY IT MATTERS

A single debit line or a single credit line can never pass balance validation.

TEST COVERAGE

Minimum line count and balance cases.

G-09 MEDIUM

Date Format Is Not Clearly Defined

WHAT IS UNCLEAR

Excel dates are locale-sensitive. A text date like `05/06/2026` can mean different dates in different regions.

ASSUMPTION

Accept real Excel date values or `DATE()` formulas and normalize them to YYYY-MM-DD. Text dates should be rejected unless a locale rule is documented.

WHY IT MATTERS

Silent date conversion may create a journal in the wrong accounting date.

TEST COVERAGE

Date format and cross-platform date cases.

G-10 HIGH

Reporting Period Is Not Clearly Defined

WHAT IS UNCLEAR

The spec uses `reportingPeriod` or `postPeriod`, but it does not explain whether this is a period code, a month, a date range, or an ERP financial period.

ASSUMPTION

`postPeriod` should be an ERP financial period, for example `05-2026`. The system should validate that `transactionDate` is inside this period.

WHY IT MATTERS

A period is not just one date. It has a start and end date, and the system must know them.

TEST COVERAGE

Period validation and date-period consistency cases.

G-11 HIGH

Period Start And End Dates Are Not Described

WHAT IS UNCLEAR

The spec does not say how the system knows the start and end date of the selected period.

ASSUMPTION

The system should get period start and end dates from ERP master data and validate `transactionDate` against them.

WHY IT MATTERS

Without period boundaries, the system cannot safely check whether a transaction date belongs to the selected period.

TEST COVERAGE

Date-period validation cases.

G-12 HIGH

Closed Period Behavior Is Not Defined

WHAT IS UNCLEAR

The spec does not say what happens if the selected `postPeriod` is closed or locked in ERP.

ASSUMPTION

Creating a draft in a closed or locked period should be rejected with a clear Excel error.

WHY IT MATTERS

Writing financial data to a closed period can break accounting controls.

TEST COVERAGE

Closed period cases.

G-13 HIGH

Allowed Ledger Values Are Not Listed

WHAT IS UNCLEAR

The spec says ledger is required, but it does not say which values are valid. It is unclear whether the user should enter ledger code, ledger name, or ERP internal ID.

ASSUMPTION

Ledger code is the safest user-facing assumption unless the product confirms another value.

WHY IT MATTERS

Users may enter the wrong value if the accepted ledger format is not clear.

TEST COVERAGE

Ledger validation and permission cases.

G-14 HIGH

Account Identifier Is Not Defined

WHAT IS UNCLEAR

The spec says account is required, but it does not say whether the user should enter account code, account name, or ERP internal ID.

ASSUMPTION

The user should enter the ERP account code. The system should validate that the account exists, is active, and is available to the current user.

WHY IT MATTERS

Account names can be duplicated or translated. Internal IDs may not be visible to business users.

TEST COVERAGE

Account validation cases.

G-15 HIGH

Account Validation Rules Are Missing

WHAT IS UNCLEAR

The spec does not describe how invalid, inactive, restricted, or missing accounts should be handled.

ASSUMPTION

The system should reject accounts that do not exist, are inactive, or are not available to the user.

WHY IT MATTERS

Wrong accounts lead to wrong financial data and are hard to fix later.

TEST COVERAGE

Account required, invalid account, inactive account, and restricted account cases.

G-16 HIGH

Allowed Currencies For Each Ledger Are Not Defined

WHAT IS UNCLEAR

The spec says currency can default from the ledger, but it does not say which currencies are allowed for each ledger.

ASSUMPTION

The system should validate currency against the selected ledger. If currency is empty, the system should use the ledger default currency.

WHY IT MATTERS

A ledger may not allow every currency. A wrong ledger and currency combination should not create a batch.

TEST COVERAGE

Currency and ledger combination cases.

G-17 HIGH

Multi-Currency Behavior Is Not Defined

WHAT IS UNCLEAR

The spec does not say whether one batch can include lines with different currencies.

ASSUMPTION

All lines in one batch should use the same currency unless ERP conversion rules are clearly documented.

WHY IT MATTERS

Balance between different currencies is not meaningful without conversion rules.

TEST COVERAGE

Currency validation and grouping cases.

G-18 MEDIUM

Branch Behavior Is Not Defined

WHAT IS UNCLEAR

Branch is optional, but the spec does not say whether branch is batch-level or line-level, whether it is validated, or whether it affects grouping.

ASSUMPTION

Treat branch as a batch-level field. If it is provided, validate it and include it in the grouping key.

WHY IT MATTERS

Lines from different branches should not be mixed unless the ERP explicitly allows it.

TEST COVERAGE

Branch validation and grouping cases.

G-19 CRITICAL

Atomicity Behavior Is Missing

WHAT IS UNCLEAR

The spec does not say what happens if one line fails validation or ERP fails in the middle of creation.

ASSUMPTION

Use all-or-nothing behavior. If any line or batch fails, nothing should be written to ERP.

WHY IT MATTERS

Partial journal creation can leave wrong or incomplete financial data.

TEST COVERAGE

Atomicity and partial failure cases.

G-20 HIGH

Partial Success Across Multiple Batches Is Not Defined

WHAT IS UNCLEAR

The selected range may create several batches. The spec does not say what happens if one batch is valid and another one is invalid.

ASSUMPTION

The safest default is all-or-nothing for the whole Writeback operation.

WHY IT MATTERS

Partial success can confuse users and make reconciliation harder.

TEST COVERAGE

Mixed valid and invalid group cases.

G-21 HIGH

Permission Model Is Not Described

WHAT IS UNCLEAR

The spec does not define which ERP permissions are needed to create draft journals, use ledgers, use branches, or use accounts.

ASSUMPTION

ERP permissions should be enforced server-side. If access is missing, Excel should show a clear permission error and no batch should be created.

WHY IT MATTERS

Financial write access must not depend only on Excel-side checks.

TEST COVERAGE

Permission cases.

G-22 HIGH

User-Facing Excel Error Behavior Is Not Described

WHAT IS UNCLEAR

The spec does not say where and how errors are shown in Excel. It is unclear whether errors appear in the formula cell, task pane, notification, or result area.

ASSUMPTION

Excel should show a clear user-visible error with the affected field and row or batch group.

WHY IT MATTERS

Excel is the main user interface. Users must understand what failed and where.

TEST COVERAGE

Required field, invalid master data, balance, and technical error cases.

G-23 MEDIUM

Success Result Per Excel Row Is Not Clear

WHAT IS UNCLEAR

The spec says the function returns a message like `Draft batch #JE-00087 created`, but it does not say what happens when several Excel rows are grouped into one batch.

ASSUMPTION

All rows included in the same batch should show the same batch number, or Excel should show one clear grouped result with affected rows.

WHY IT MATTERS

Users need to understand which rows were included in the created batch.

TEST COVERAGE

Success result and grouped batch cases.

G-24 HIGH

Selected Excel Range Behavior Is Not Clear

WHAT IS UNCLEAR

The spec does not explain how the add-in decides which Excel rows are included in Writeback.

ASSUMPTION

Only the selected or configured range should be processed. Empty, hidden, filtered, and invalid rows should follow documented rules.

WHY IT MATTERS

Workbooks can contain empty rows, hidden rows, filtered rows, copied formulas, and extra data outside the selected range.

TEST COVERAGE

Selected range processing cases.

G-25 HIGH

Cross-Platform Excel Behavior Is Not Defined

WHAT IS UNCLEAR

The spec does not describe expected behavior in Excel for Windows, Excel for macOS, and Excel Web.

ASSUMPTION

Behavior should be the same across supported platforms, or limitations should be clearly documented.

WHY IT MATTERS

Dates, decimals, add-in behavior, and formula recalculation can differ by platform.

TEST COVERAGE

Cross-platform cases.

G-26 HIGH

API Contract Is Not Described

WHAT IS UNCLEAR

The spec does not define the API payload, response schema, field mapping, or error code mapping between Excel, backend, and ERP.

ASSUMPTION

The product should document request fields, response fields, required fields, data types, and error format.

WHY IT MATTERS

Without a contract, API tests are based on guesses instead of confirmed behavior.

TEST COVERAGE

API contract and request/response mapping cases.

G-27 HIGH

Audit Trail Is Not Specified

WHAT IS UNCLEAR

The spec does not say what is recorded in the audit trail for successful and failed Writeback operations.

ASSUMPTION

Each operation should be auditable: user, timestamp, source, result, error code, and created batch number when available.

WHY IT MATTERS

Support needs traceability to investigate duplicates, failed operations, and wrong ERP data.

TEST COVERAGE

Audit trail cases.

G-28 MEDIUM

Error Alerting Is Not Described

WHAT IS UNCLEAR

The spec does not say whether repeated or critical failures create internal alerts (e.g. in Sentry, Slack, or monitoring tools).

ASSUMPTION

Critical failures such as ERP unavailable, timeout, or repeated Writeback failures should trigger an internal alert.

WHY IT MATTERS

Teams should know about production failures before many users report them.

TEST COVERAGE

Monitoring and technical failure cases.

G-29 MEDIUM

Field Limits And Numeric Precision Are Not Specified

WHAT IS UNCLEAR

The spec does not define maximum lengths for text fields or numeric precision for debit and credit.

ASSUMPTION

Define limits for each text field and amount field. Reject values that exceed limits or document safe truncation rules.

WHY IT MATTERS

Undefined limits can cause truncation, overflow, rounding issues, or ERP errors.

TEST COVERAGE

Boundary value cases.

G-30 MEDIUM

Sensitive Data Logging Rules Are Not Defined

WHAT IS UNCLEAR

The spec does not say which request data can be logged and which data must be masked.

ASSUMPTION

Logs should include useful support data but must not include tokens, passwords, or sensitive connection details.

WHY IT MATTERS

Logs must help support without leaking secrets.

TEST COVERAGE

Telemetry and log safety cases.

G-31 MEDIUM

Preview Before Writeback Is Not Described

WHAT IS UNCLEAR

The spec does not say whether the user can preview the batch before it is created in ERP.

ASSUMPTION

If preview exists, it should show grouped batches, debit total, credit total, ledger, period, currency, and line count before final submission.

WHY IT MATTERS

This is a financial write operation. A preview helps users catch mistakes before sending data to ERP.

TEST COVERAGE

Preview or pre-submit validation cases if the product supports this feature.

G-32 HIGH

Duplicate Business Data Rule Is Not Defined

WHAT IS UNCLEAR

The spec does not say what happens if the user sends the same journal data again as a new operation (new idempotency key, same business content).

ASSUMPTION

The product should define whether duplicate business data is allowed, blocked, or shown as a warning.

WHY IT MATTERS

Idempotency protects retries, but it does not answer whether the same journal can be submitted again intentionally or by mistake.

TEST COVERAGE

Duplicate business data cases.

04 GLOSSARY

Parameter, accounting and testing vocabulary used in this assessment

All terms used in test cases and gap analysis are defined here. This makes the document self-contained - no ERP background required to read a test case.

BATCH INPUT Function parameters - batch level

`connectionName`	Name of the ERP connection the add-in writes to; selects which ERP instance / company receives the batch.
`transactionDate`	The accounting date of the journal batch; must fall inside the post period.
`postPeriod`	The financial period the batch posts to (e.g. 05-2026); must be open and consistent with the transaction date.
`batchDescription`	Optional text describing the whole batch; also part of the grouping key.
`branch`	Optional company branch the batch belongs to; usually tied to the ledger.
`ledger`	The target GL ledger the batch is written to; defines the default currency.

LINE INPUT Function parameters - line level

`currency`	Currency code for the line; defaults to the ledger currency when empty.
`account`	The GL account the line posts to; must exist and be active in the ERP.
`debit`	Amount on the debit side of a line; a line carries exactly one of debit or credit.
`credit`	Amount on the credit side of a line; a line carries exactly one of debit or credit.
`lineDescription`	Optional text describing a single journal line.

ACCOUNTING Core financial concepts

GL (General Ledger)	The central record of all financial transactions; financial reports are built from it.
Journal / journal entry	A recorded financial transaction made of one or more lines that must balance.
Journal line	One row of a journal: an account with a debit OR a credit amount.
Batch	A group of journal lines that share the same batch fields (date, ledger, description).
Double-entry	Each transaction posts equal debits and credits across accounts.
Balance / balanced	Total debits equal total credits across all lines of the batch.
Draft batch	A journal batch that is created but not yet posted; sits in On Hold for review.
On Hold	The status of a draft batch awaiting manual review and posting; never posted by the function.
Post / Release	The manual action that finalizes a draft into the ledger; performed only by a human.
Reporting period	A predefined accounting period that a transaction date belongs to.
Master data / dictionary value	Predefined ERP reference data: connections, ledgers, accounts, branches, currencies, periods.

QA Testing and reliability concepts

Writeback	The documented action that sends the prepared range from Excel to the ERP.
Grouping key	The set of fields (connection, date, period, ledger, branch, currency, description) that decides which lines form one batch.
Idempotency	Repeating the same operation does not create a duplicate batch; one logical operation = one result.
Idempotency key / operation id	A stable identifier used to recognize a repeated operation and prevent duplicates.
Atomicity	All-or-nothing: either the whole batch is created or nothing is; never a partial batch.
Partial batch	An incomplete write (some lines saved, some not); must never happen.
Side effect	An unintended ERP write caused by Excel actions (recalculation, copy, open) without an explicit Writeback.
Equivalence classes	Grouping inputs that behave the same (valid, zero, negative, text) so one representative of each is tested.
Boundary values	Inputs at the edges of valid ranges (min, max, max precision) where bugs often hide.
Normalization	Cleaning input to a canonical form (trim spaces, fix letter case) per a documented rule.
Poison message	A malformed message in a queue that must not block processing of valid messages.
Dead-letter handling	Routing a repeatedly failing message aside so the queue keeps working.
Correlation / operation id	An identifier that lets one operation be traced across Excel, API, and ERP in logs.
Telemetry	Logs and metrics that record what happened, used for tracing and troubleshooting.

CASES

Terms introduced in the test cases

> These phrasings appear in test case titles and expected results. Defining them keeps the document self-consistent.

`On-Hold draft / draft On Hold`	Same as a draft batch; emphasizes that the function only ever produces an unposted, review-ready batch.
`Self-cancelling entry`	A balanced line pair that debits and credits the same account for the same amount; passes balance validation but is economically meaningless (left to human review).
`Documented rule / per documented rule`	Behavior that the spec must define (e.g. trimming, rounding, defaulting); the case checks the implementation against whatever that rule turns out to be.
`Round-trip`	The full path Excel → API/ERP → response back to Excel; used when a check requires confirmation from the ERP, not just the cell.
`All-or-nothing`	Plain-language phrasing of atomicity used in the expected results.

05

AI REFLECTION

Transparent account of assisted work and human review

// SECTION 01

01

Input Materials Used During the Analysis

Several sources fed the analysis. The spec defined what the function should do; my research and notes added context, open questions, and edge cases that the spec did not cover.

01. The original WRITEBACKJOURNALDRAFT specification
02. My own research into ERP systems, GL journals, writeback flows, and financial terminology
03. Google Docs notes - questions, draft ideas, and risks I spotted while reading the spec
04. Miro board - end-to-end flow, unclear behaviors, edge cases, and missing rules
05. AI tools used during research and drafting: ChatGPT, Claude, and Codex
06. AI-assisted explanations of how similar systems may work under the hood
07. My own QA thinking on Excel behavior, ERP side effects, financial risk, and integration edge cases

// SECTION 02

02

How I Used AI

Honestly, AI saved me the mechanical first-draft work and let me start reacting to something concrete instead of a blank page. I leaned on it most for these things:

WHAT I LEANED ON AI FOR

01. Getting up to speed on a domain I didn't know well - ERP, GL journals, ledgers, periods, debit/credit
02. Seeing the feature as an end-to-end flow (Excel - add-in - API - ERP - draft batch - result), not just a formula
03. Brainstorming test categories and surfacing early risks before I wrote anything
04. Drafting a first version of the cases and gap descriptions in bulk
05. Merging near-duplicates and making the suite leaner
06. Improving wording and turning raw notes into a readable artifact - including phrasing my thinking in clear English

I didn't take any output at face value: each result was kept, rewritten, or thrown out. AI just gave me something to react to. Concretely, it saved up to an hour of boilerplate - time that went into the parts that actually needed thinking.

// PROMPT LOG

FIVE PROMPTS, IN ORDER OF USE

// PROMPT 01 OF 05

03 Domain & System Understanding

> PROMPT USED

```
I need to test a custom Excel function called WRITEBACKJOURNALDRAFT.

Before writing test cases, explain how similar Excel-to-ERP writeback
systems usually work. Explain ERP, GL journal batch, journal line,
ledger, account, postPeriod, debit, and credit in simple terms.

Show the possible flow: Excel - add-in - API - ERP - draft batch - result.

What risks should a QA engineer think about before writing test cases?
```

+ WHAT WORKED

- Made me see the feature as a system flow, not just an Excel formula
- Explained ERP terms in plain language - ledger, account, period, journal line
- Separated user-entered fields from generated technical fields
- Surfaced early risks: duplicate creation, wrong status, timeout and retry

- WHAT FELL SHORT

- Explained generic ERP concepts without tying them to the spec
- Presented some assumptions as if they were confirmed behavior
- Needed extra prompting to move from theory to testability

> WHAT I DID WITH IT

- Tied the explanations back to the actual WRITEBACKJOURNALDRAFT fields
- Turned unclear points into gap entries instead of accepting them
- Kept the research as background context, not as final spec

NEXT PROMPT

// PROMPT 02 OF 05

04 Initial Test Brainstorming

> PROMPT USED

```
I'm a QA engineer writing test cases for WRITEBACKJOURNALDRAFT.

Each call defines one journal line. Multiple calls form one batch.
The batch must always be On Hold. It must not post automatically.

Here is the full spec: [pasted]

Give me a complete list of test categories and flag ambiguities,
missing rules, or risky assumptions.

Focus on: batch grouping, balance, required fields, ERP side effects,
idempotency, retry/timeout, permissions, audit, errors, cross-platform.
```

+ WHAT WORKED

- Mapped the main areas quickly: happy path, validation, balance, status, errors
- Spotted the naming mismatch (**date/reportingPeriod** vs **transactionDate/postPeriod**)
- Suggested decimal-precision risks - important for financial data
- Treated the function as a system integration, not just a formula

- WHAT FELL SHORT

- Took "at least one line required" literally - a single debit can't balance
- Didn't challenge the grouping key (branch? currency? postPeriod?)
- Assumed all cells return the same batch number - the spec doesn't confirm it
- Some suggested cases were too generic, with no concrete test data

> WHAT I DID WITH IT

- Added gap analysis for grouping, return-value scope, and range behavior
- Rewrote vague assumptions as explicit gaps linked to test coverage
- Simplified the language so the gaps are useful to QA, devs, and product

NEXT PROMPT

// PROMPT 03 OF 05

05 Formal Test-Case Generation

> PROMPT USED

```
Write formal test cases using this structure:
TC-ID - Category - Title - Preconditions - Test Data - Steps - Expected Result.

No Priority or Severity. Simple English. Numbered lists for
Preconditions, Steps, Expected Result. Preconditions specific to each
case, not a shared header. Group duplicates. Smoke tests at the end.
```

+ WHAT WORKED

- Produced a clean, structured suite quickly
- Covered the main flows: creation, On Hold, no auto-post, required fields, balance, dedupe
- Consistent structure across all cases with separate columns

- WHAT FELL SHORT

- First version had too many near-duplicates - needed manual merging
- Some preconditions were still vague ("valid ERP data exists")
- Verification steps were mixed into Steps instead of Expected Result

> WHAT I DID WITH IT

- Merged near-duplicates into grouped scenarios
- Rewrote preconditions to be specific per case
- Moved all verification into Expected Result; removed Priority/Severity

NEXT PROMPT

// PROMPT 04 OF 05

06 Gap-Analysis Improvement

> PROMPT USED

```
Review the gaps and assumptions as a senior QA analyst.
Make them easier to understand, no complex English.
Add missing gaps from my Miro notes.

Structure:
Gap - What is unclear - Assumption - Why it matters - Test coverage.

Add your own gaps if something important is missing.
```

+ WHAT WORKED

- Restructured the gaps into a cleaner, more readable format
- Connected each gap to practical test coverage
- Added missing areas: Excel error behavior, alerting, range edge cases, duplicate business data, allowed currencies per ledger, partial success, validation order, and sensitive data in logs

- WHAT FELL SHORT

- Sometimes over-explained in technical wording that added no value
- Still framed a few assumptions as confirmed facts

> WHAT I DID WITH IT

- Removed wording that didn't help testing or requirement clarity
- Kept each gap focused on business risk, not just a technical detail
- Removed assumptions that were presented as facts

NEXT PROMPT

// PROMPT 05 OF 05

07 Field Specification Review

> PROMPT USED

```
Evaluate this field specification table and make it clearer.

Fields: connectionName, transactionDate, postPeriod, batchDescription,
branch, ledger, currency, account, debit, credit, lineDescription,
externalLineId, idempotencyKey, correlationId.

Review as a senior QA analyst. Clarify format, API mapping, validation,
defaults, limits. Add columns if needed.
```

+ WHAT WORKED

- Found fields with unclear definitions
- Flagged that **ledger** and **account** shouldn't accept code/name/ID unless the spec confirms all three
- Suggested a Validation Owner column (Add-in / Backend / ERP)
- Clarified that generated fields must not be user-entered

- WHAT FELL SHORT

- Left some limits as if confirmed, when they were only assumptions

> WHAT I DID WITH IT

- Made the table stricter - removed vague "code/name/ID" for one canonical form
- Marked unconfirmed limits as assumptions, not facts
- Added global validation rules below the table for shared constraints

// SECTION 08

08 Where AI Was Not Enough

ACROSS EVERY PROMPT, AI WAS CONSISTENTLY WEAKER AT

01. Challenging the spec unless explicitly told to
02. Detecting hidden product risks without direct guidance
03. Avoiding assumptions when the spec left something open
04. Deciding which cases to merge versus keep separate
05. Treating Excel recalculation as dangerous when the formula has real ERP side effects

The pattern: AI reads a specification charitably. It fills gaps with plausible assumptions instead of flagging them as risks. The most valuable part of this task was the opposite instinct - reading carefully and asking, every time: does this actually make sense?

// SECTION 09

09 Why I Decided to Create a Website

As the analysis grew, one flat document became too overloaded. There were test cases, gaps, assumptions, field specifications, validation rules, AI usage notes, and review comments.

Because of that, I decided that a small structured website would be a better format. It would let the reader switch between sections instead of scrolling through one long document.

Looking back now, there are things I would still rewrite with more time. Either way, I really enjoyed working in a new domain. Thank you.

06

TEST CASES

Execution-ready workbook supplied separately

// SEPARATE DELIVERABLE

TEST CASES WORKBOOK

The full test case suite is provided as a separate Excel workbook so that areas, test data, preconditions, steps and expected results remain easy to filter, scan and execute.

XLSX WRITEBACKJOURNALDRAFT_Test_Cases.xlsx